Day 7: File Handling, Data Formats, Serialization & Relational Databases

Welcome to Day 7! Today we explore the mechanisms Python uses to persist, format, serialize, and query data. Rather than just memorizing boilerplate scripts, we will focus on understanding the **core functions, method signatures, parameter mechanics, and architectural concepts** that power Python's file I/O, structured format parsers, object serialization, and relational database drivers.

Table of Contents

1. Part 1: File I/O Streams & Context Managers
2. Part 2: Structured Tabular Formats (`csv` Module)
3. Part 3: Hierarchical Serialization (`json` Module)
4. Part 4: Object Serialization & Binary Persistence (`pickle` Module)
5. Part 5: Relational Databases & SQLite (Python DB-API 2.0 / `sqlite3`)

Part 1: File I/O Streams & Context Managers

1. The File Stream Architecture

When Python interacts with a file on disk, it does not directly manipulate the storage hardware. Instead, the Operating System allocates an **I/O Stream** and a **File Descriptor** (an integer handle in the OS kernel table). Python wraps this descriptor in a high-level file object that maintains:

- A **Stream Position Pointer** (cursor offset indicating where the next byte/character will be read or written).
- An **Internal I/O Buffer** (reducing expensive physical disk writes by batching data in memory).
- A **Character Encoding Decoder** (e.g., UTF-8 translation between raw bytes and Python `str` Unicode codepoints).

---

2. Main Functions & Methods in File I/O

The `open()` Constructor Function

```
file_object = open(file, mode='r', buffering=-1, encoding=None,
errors=None, newline=None)
```

- **`file`**: String path (or `pathlib.Path`) to the target file.
- **`mode`**: Access mode specifying stream permissions and pointer placement:
  - `'r'` (*Read*): Opens existing file for reading from byte offset `0`. Raises `FileNotFoundError` if absent.

    - `'w'` (*Write*): Opens for writing. Truncates (erases) file to 0 bytes if it exists, or creates a new file.
    - `'a'` (*Append*): Opens for writing with stream pointer at the end of the file. Preserves existing data.
    - `'r+'` (*Read & Write*): Opens existing file for both reading and writing without automatic truncation.
    - `'b'` (*Binary Mode*): Disables automatic Unicode encoding/decoding, returning raw `bytes` (e.g. `'rb'`, `'wb'`).
- **`encoding`**: Character encoding standard. **Always specify `encoding="utf-8"`** to ensure cross-platform consistency between macOS, Linux, and Windows.
- **`newline`**: Controls universal newline translation (`\n` vs `\r\n`). When writing CSVs, setting `newline=''` is mandatory to prevent blank lines on Windows.

Core Stream Reading Methods

Method	Signature	Return Type	Operational Behavior
`read()`	`f.read(size=-1)`	`str` / `bytes`	Reads the entire file content into a single string (or up to `size` characters/bytes if specified).
`readline()`	`f.readline(size=-1)`	`str` / `bytes`	Reads the next single line up to the newline character `\n`. Returns `""` (empty string) upon reaching EOF (End of File).
`readlines()`	`f.readlines(hint=-1)`	`list[str]`	Reads all remaining lines and returns them as a list of strings.
Direct Iteration	`for line in f:`	Generator `str`	**Best Practice**: Streams lines lazily into memory one line at a time. Ideal for massive (multi-gigabyte) files.

Core Stream Writing & Positioning Methods

- **`f.write(string)`**: Writes a string to the stream buffer and returns the integer count of characters written. It does **not** append an automatic newline (`\n`).
- **`f.writelines(iterable)`**: Writes a sequence of strings (e.g., a list of lines) to the stream. Does not add line separators.
- **`f.tell()`**: Returns the current integer byte offset of the stream cursor.
- **`f.seek(offset, whence=0)`**: Moves the stream cursor to a new position:
    - `whence=0` (*default*): Absolute offset from the beginning of the file.
    - `whence=1`: Relative offset from the current stream position.
    - `whence=2`: Relative offset from the end of the file (typically used with negative offsets in binary mode).

- `f.flush()`: Forces immediate flushing of the internal Python write buffer to the OS disk buffer without closing the stream.
- `f.close()`: Flushes buffers and releases the operating system file descriptor handle.

---

3. Context Managers & The `with` Statement Protocol

Manual file handling requires explicit `try...finally` blocks to ensure `f.close()` executes even during runtime crashes. The `with` statement utilizes Python's Context Manager protocol:

- Upon entering the block, Python executes `f.__enter__()`, returning the file object.
- Upon exiting the block (normally or via an unhandled exception), Python automatically invokes `f.__exit__(exc_type, exc_val, exc_tb)`, guaranteeing that the stream closes immediately.

```
# Concise Context-Managed File Operations
with open("system_log.txt", "w", encoding="utf-8") as f:
    f.write("Line 1: System Boot\nLine 2: Ready\n")

# Reading lazily line by line
with open("system_log.txt", "r", encoding="utf-8") as f:
    for line in f:
        print("Log Entry:", line.strip())
```

Part 2: Structured Tabular Formats (`csv` Module)

The standard `csv` module parses delimited tabular text files without requiring manual `.split(",")` operations, properly handling quoted fields, commas inside text, and escaped newlines.

1. Main Functions & Classes in `csv`

A. Positional Row Processing: `csv.reader` & `csv.writer`

- `csv.reader(csvfile, dialect='excel', **fmtparams)`:
  - Returns an iterator that parses each line into a **list of strings**.
  - Key parameters: `delimiter=','` (column separator), `quotechar='"'` (quoting character).
- `csv.writer(csvfile, dialect='excel', **fmtparams)`:
  - Returns a writer object responsible for converting sequences into delimited strings.
  - `writer.writerow(row_sequence)`: Writes a single row list/tuple.
  - `writer.writerows(list_of_rows)`: Writes multiple rows in batch.

B. Dictionary-Based Column Mapping: `csv.DictReader` & `csv.DictWriter`

- `csv.DictReader(f, fieldnames=None, restkey=None, restval=None)`:
  - Reads tabular data directly into Python dictionaries (`dict`).
  - If `fieldnames` is omitted, the first row of the CSV is automatically consumed as dictionary keys.
  - Each subsequent row maps column headers to corresponding row string values.

- `csv.DictWriter(f, fieldnames, restval='', extrasaction='raise')`:
  - Writes dictionary mappings into CSV rows based on the prescribed `fieldnames` list.
  - `writer.writeheader()`: Writes the header row containing the keys listed in `fieldnames`.
  - `writer.writerow(row_dict)`: Writes a dictionary where keys match `fieldnames`.

```
import csv

# Writing CSV via DictWriter
records = [{"id": 1, "product": "Chai", "price": 18.0}, {"id": 2,
"product": "Chang", "price": 19.0}]
with open("products.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["id", "product", "price"])
    writer.writeheader()
    writer.writerows(records)

# Reading CSV via DictReader
with open("products.csv", "r", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        print(f"Product: {row['product']} | Price:
${float(row['price']):.2f}")
```

Part 3: Hierarchical Serialization (`json` Module)

JSON (JavaScript Object Notation) is a lightweight, human-readable text format for hierarchical data exchange. Python's standard `json` module translates between JSON types and Python native types.

1. Data Type Mapping

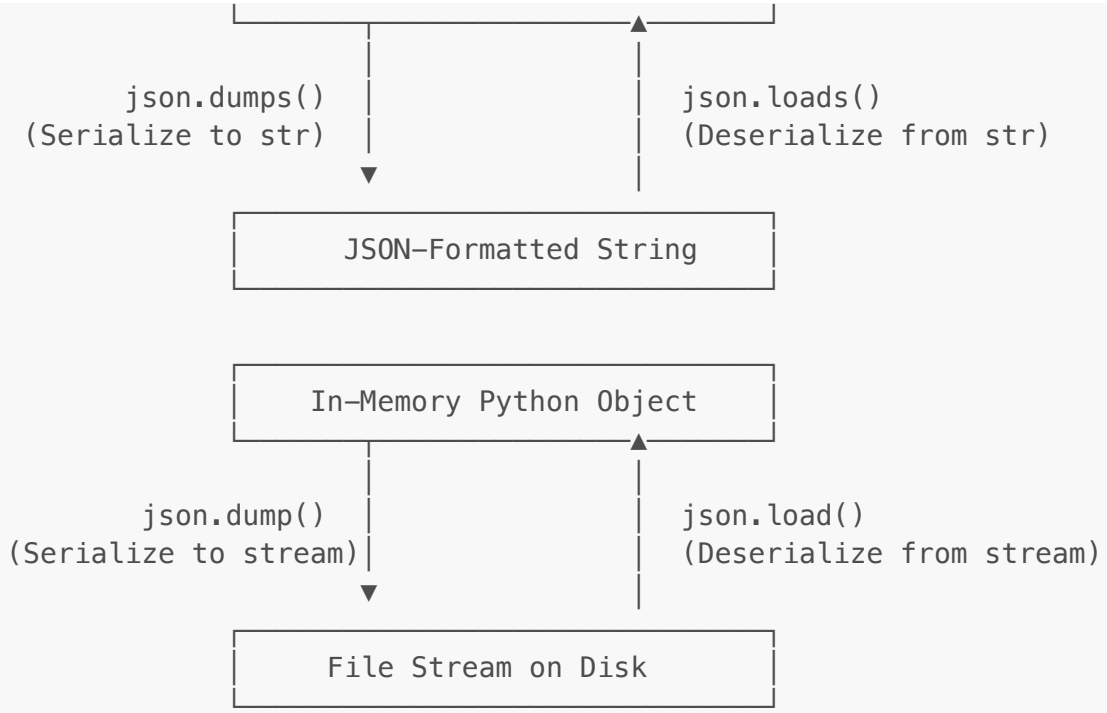
JSON Data Type	Python Native Equivalent
`object` (`{"key": "value"}`)	`dict`
`array` (`[1, 2, 3]`)	`list`
`string` (`"hello"`)	`str`
`number (int / real)`	`int` / `float`
`boolean` (`true` / `false`)	`bool` (`True` / `False`)
`null`	`None`

---

2. The Four Core JSON Functions Matrix

The `json` module is built around **four fundamental functions**, divided into **string conversions** (functions ending in `s`) and **file stream conversions**:

```
┌──────────────────────────────────┐
│    In-Memory Python Object       │
```



Function 1: `json.dumps(obj, *, indent=None, sort_keys=False, default=None)`

- **Purpose**: Serializes in-memory Python object `obj` into a formatted JSON **string** (`str`).
- `indent`: Integer indentation level for human-readable pretty-printing (e.g. `indent=4`).
- `sort_keys`: If `True`, sorts dictionary keys alphabetically.
- `default`: A fallback callable for encoding custom objects that are not natively serializable.

Function 2: `json.loads(s, *, parse_float=None, parse_int=None)`

- **Purpose**: Deserializes a JSON **string** `s` back into native Python dictionaries/lists.
- Raises `json.JSONDecodeError` if the string contains malformed JSON syntax.

Function 3: `json.dump(obj, fp, *, indent=None, sort_keys=False)`

- **Purpose**: Serializes Python object `obj` and writes it directly to an open text file stream `fp`.

Function 4: `json.load(fp)`

- **Purpose**: Reads directly from an open text file stream `fp` and parses JSON into a Python data structure.

```
import json

payload = {"order_id": 10248, "customer": "VINET", "items": [{"id": 11,
"qty": 12}]}

# 1. To String (dumps) & From String (loads)
json_str = json.dumps(payload, indent=2)
restored_obj = json.loads(json_str)
```

```
# 2. To File (dump) & From File (load)
with open("order.json", "w", encoding="utf-8") as f:
    json.dump(payload, f, indent=4)

with open("order.json", "r", encoding="utf-8") as f:
    data_from_file = json.load(f)
```

Part 4: Object Serialization & Binary Persistence (`pickle` Module)

1. What is Pickling?

While JSON only represents generic data types (strings, numbers, lists, dictionaries), Python applications often need to persist **exact in-memory Python objects**—including custom class instances, function references, and recursive data structures.

Pickling (*Object Serialization*) converts a Python object hierarchy into a byte stream (`bytes`), which can be stored on disk or transmitted over a network. **Unpickling** reconstructs the exact Python object back in memory.

---

2. The Four Core Pickle Functions Matrix

Similar to `json`, the `pickle` module provides two string/byte functions and two stream functions:

Function	Input	Output	Operational Behavior
`pickle.dumps(obj)`	Python object	`bytes` object	Serializes object into an in-memory binary byte stream.
`pickle.loads(bytes_data)`	`bytes` object	Python object	Deserializes an in-memory byte buffer back into a live Python object.
`pickle.dump(obj, file)`	Object + File stream	None (writes to disk)	Serializes object directly to an open binary file (`'wb'`).
`pickle.load(file)`	Binary file stream	Python object	Reads byte stream from binary file (`'rb'`) and reconstructs the object.

```
import pickle

class ProductCatalog:
    def __init__(self, category):
        self.category = category
        self.items = []

    def add_product(self, name, price):
        self.items.append({"name": name, "price": price})

catalog = ProductCatalog("Beverages")
```

```
catalog.add_product("Chai", 18.0)

# Save live class instance to binary file (dump)
with open("catalog.pkl", "wb") as f:
    pickle.dump(catalog, f)

# Restore live class instance from binary file (load)
with open("catalog.pkl", "rb") as f:
    restored_catalog = pickle.load(f)

print(f"Restored Category: {restored_catalog.category} | Items:
{restored_catalog.items}")
```

---

3. What Can and Cannot Be Pickled?

Supported Types:

- Built-in primitives: `None`, booleans, integers, floats, complex numbers, strings, bytes.
- Built-in containers: `tuples`, `lists`, `sets`, `dictionaries` containing picklable objects.
- Top-level functions and built-in functions (pickled by name reference).
- Top-level classes and class instances whose `__dict__` attributes are picklable.

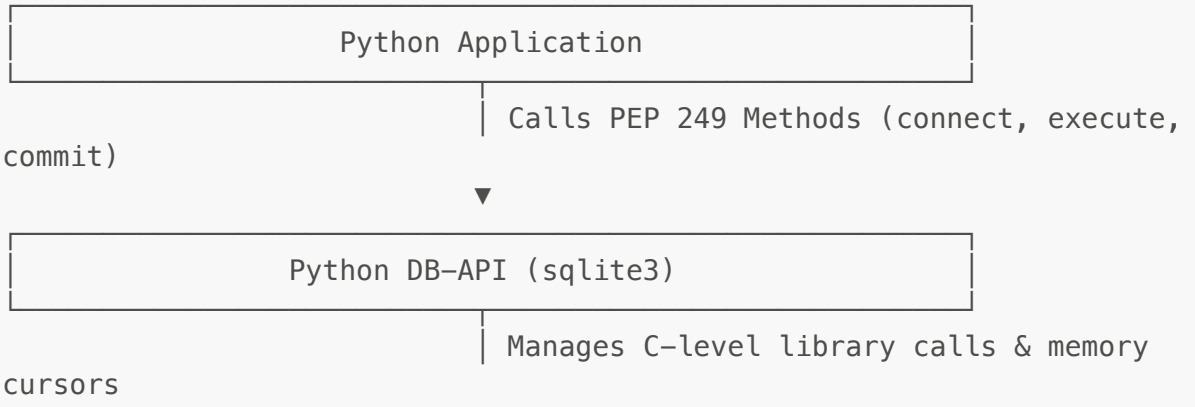
Unsupported Types:

- Open OS resources: Active file descriptors, active database connections, network sockets.
- Execution frames, generators, and running coroutines.
- Anonymous lambda functions and nested closures.

> [!CAUTION] **Pickle Security Warning**: The `pickle` format is **not secure against untrusted data**. Pickled streams can encode instructions to execute arbitrary system commands during unpickling via the `__reduce__` method. **Never unpickle untrusted data received over public networks.**

Part 5: Relational Databases & SQLite (Python DB-API 2.0 / `sqlite3`)

Python interacts with relational database management systems (RDBMS) via the **PEP 249 Database API Specification v2.0 (DB-API)**. Python includes native SQLite support via the `sqlite3` module.



```
                    ▼
┌──────────────────────────────────────────────┐
│              Embedded SQLite SQL Engine      │
│              (Database File / In-Memory)     │
└──────────────────────────────────────────────┘
```

---

1. Main Objects & Methods in `sqlite3`

Object 1: The Connection Object (`sqlite3.Connection`)

Created via `sqlite3.connect(database, timeout=5.0, ...)`:

- **`conn.cursor()`**: Instantiates and returns a new Cursor object to execute SQL commands.
- **`conn.commit()`**: Commits the current active transaction to disk storage. Required after any `INSERT`, `UPDATE`, or `DELETE`.
- **`conn.rollback()`**: Aborts the active transaction, reverting all modifications made since the last `commit()`.
- **`conn.close()`**: Closes the database connection and releases OS locks.
- **`conn.row_factory`**: Callable to customize row representations (e.g. `sqlite3.Row` allows dictionary-like column name access `row["column_name"]`).

Object 2: The Cursor Object (`sqlite3.Cursor`)

The cursor acts as a pointer and execution context for running SQL statements and retrieving result sets.

Core Execution Methods:

- **`cursor.execute(sql, parameters)`**:
  - Prepares and executes a single SQL statement.
  - **Always use parameter tuples (`?`)** instead of string concatenation.
  - Example: `cursor.execute("SELECT * FROM orders WHERE freight > ?", (50.0,))`.
- **`cursor.executemany(sql, seq_of_parameters)`**:
  - Executes a parameterized SQL command repeatedly against an iterable sequence of parameter tuples (high-speed batch inserts).
- **`cursor.executescript(sql_script)`**:
  - Executes multiple raw SQL statements separated by semicolons (e.g., initial table creation scripts).

Core Data Retrieval Methods:

- **`cursor.fetchone()`**: Retrieves the next single row tuple from the query result set, or returns `None` when exhausted.
- **`cursor.fetchmany(size)`**: Retrieves the next batch of rows as a list of tuples (up to `size` rows).
- **`cursor.fetchall()`**: Retrieves all remaining rows from the result set as a list of tuples.

Core Metadata Attributes:

- `cursor.rowcount`: Returns the number of rows modified, inserted, or deleted by the last SQL execution.
- `cursor.lastrowid`: Returns the integer primary key `id` generated by the most recent `INSERT` operation on an `AUTOINCREMENT` column.

---

2. Concise DB-API CRUD Workflow & Parameterization

```
import sqlite3

# 1. Establish Connection & Cursor
conn = sqlite3.connect("store.db")
conn.row_factory = sqlite3.Row  # Enables column-name indexing
cursor = conn.cursor()

# 2. DDL: Create Table
cursor.execute('''
CREATE TABLE IF NOT EXISTS orders (
    order_id INTEGER PRIMARY KEY,
    customer_id TEXT NOT NULL,
    freight REAL NOT NULL
)
''')

# 3. Batch Parameterized Insert (executemany)
sample_orders = [(10248, "VINET", 32.38), (10249, "TOMSP", 11.61), (10250,
"HANAR", 65.83)]
cursor.executemany("INSERT OR IGNORE INTO orders VALUES (?, ?, ?)",
sample_orders)
conn.commit()

# 4. Parameterized Query (execute + fetchall)
cursor.execute("SELECT order_id, customer_id, freight FROM orders WHERE
freight > ?", (20.0,))
for row in cursor.fetchall():
    print(f"Order #{row['order_id']} | Cust: {row['customer_id']} |
Freight: ${row['freight']:.2f}")

# 5. Clean up
conn.close()
```

> [!IMPORTANT] **Preventing SQL Injection**: Never format SQL queries with Python string formatting (e.g., `f"SELECT * FROM users WHERE name = '{user_input}'"`). Attackers can pass malicious payloads like `' OR '1'='1` to bypass security. **Always pass data as a separate tuple using `?` placeholders.**